

CloudMovie Challenge

AWS infrastructure capstone: private compute, IaC, observability, secrets, persistence, and CI/CD



1. Project goal and constraints

A small application used to demonstrate a complete cloud-engineering lifecycle

GOAL

Build a repeatable, secure, observable AWS platform

APP SCOPE

Keep Flask simple so the cloud design stays central

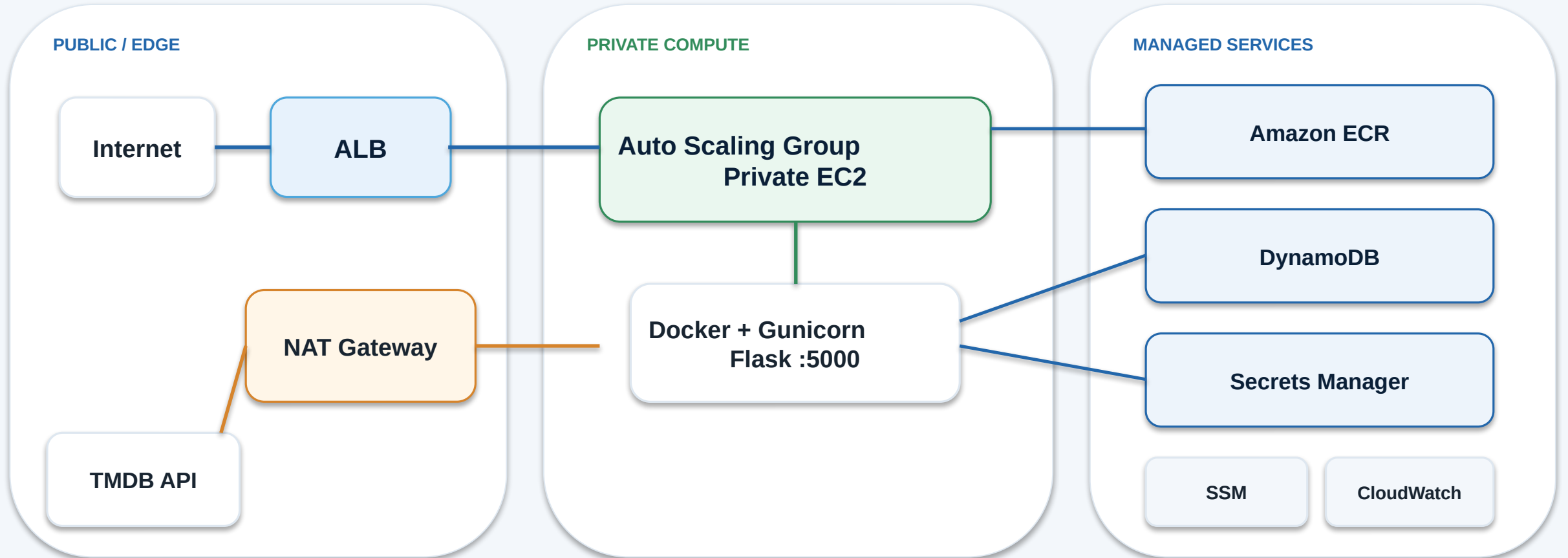
CONSTRAINT

Limited AWS credit: quality over unnecessary scale

- Infrastructure as Code instead of manual console configuration
- Private application compute with managed ingress and administration
- Persistent state, runtime secrets, monitoring, and automated deployment
- Clear trade-offs: development-sized capacity, production-oriented patterns

2. Architecture overview

Every major component has a concrete role in the request, deployment, or operations path



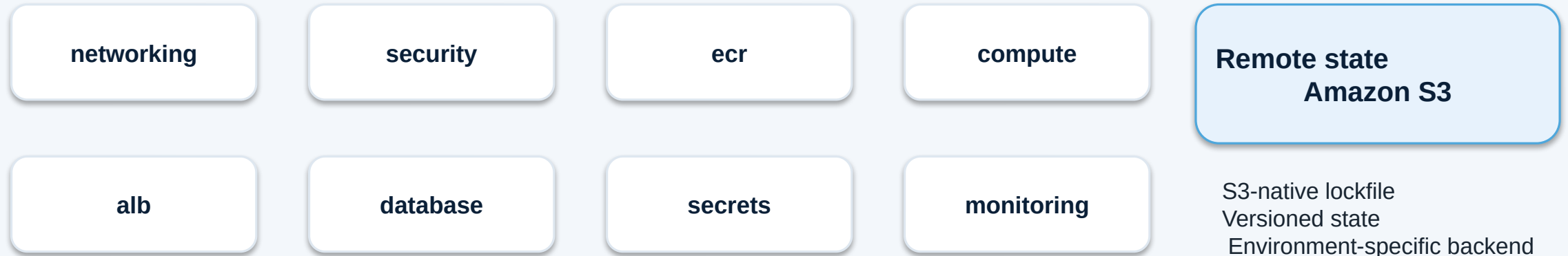
3. Networking and security

Public ingress is separated from private application compute



4. Infrastructure as Code

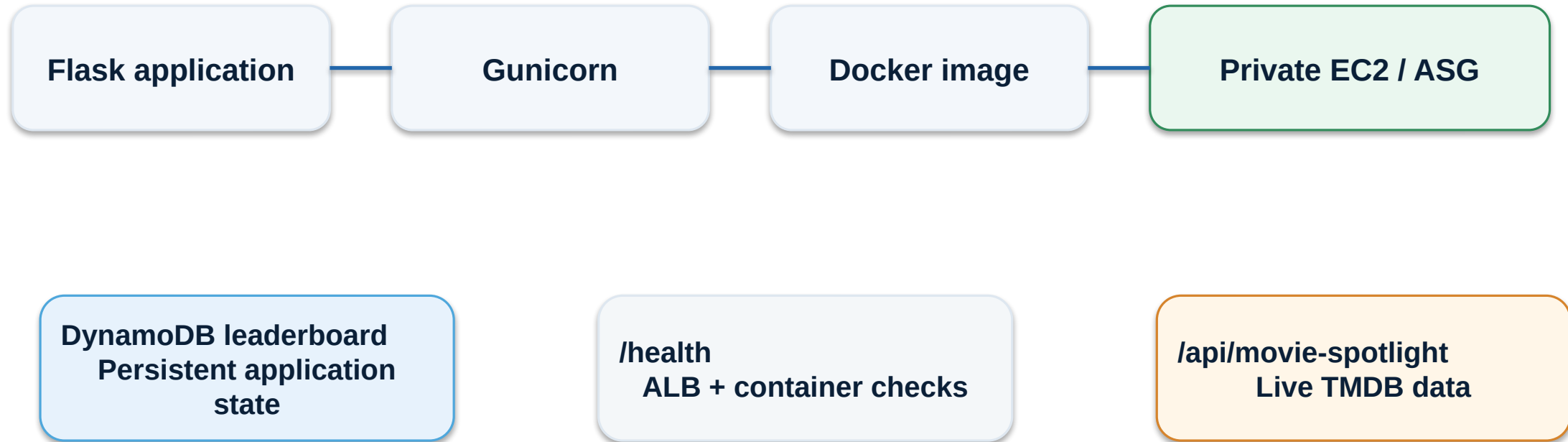
Terraform modules make the environment repeatable and auditable



- Root environment: infrastructure/environments/dev
- Saved plan before apply; module boundaries keep responsibilities clear
- Secrets value intentionally excluded from Terraform state
- Infrastructure can be destroyed and rebuilt for cost control

5. Application and persistent state

Stateless compute, managed persistence



Key design result: terminating an EC2 instance does not delete leaderboard data.

6. Secrets Manager + TMDB: why the NAT Gateway exists

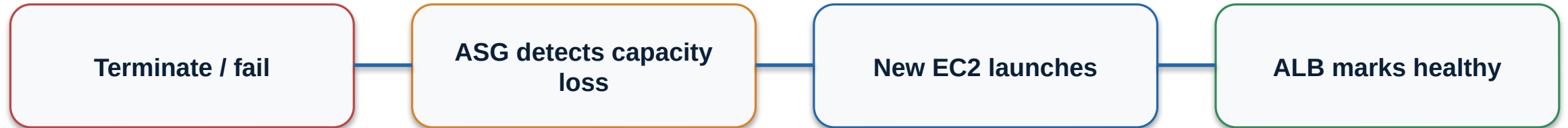
A real outbound dependency justifies private egress



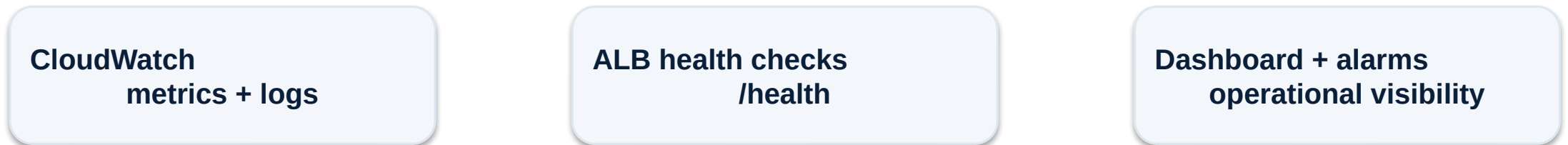
- Secret value is inserted separately - not committed and not stored in Terraform state.
- EC2 reads the secret with its IAM role at runtime.
- Private compute then reaches the external API through NAT-controlled outbound access.

7. Resilience and observability

The platform is designed to detect failure, replace compute, and expose health



Observed during testing: replacement instance pulled the image, started Docker, and registered behind the ALB.



Trade-off: desired capacity = 1 reduces cost but is not the availability target I would choose for production.

8. CI/CD: from commit to running instance

Publishing to ECR is not enough - the workflow also refreshes the runtime



- GitHub Actions authenticates to AWS and Amazon ECR.
- Application changes build a new container image and publish it to the repository.
- An Auto Scaling Group instance refresh causes replacement compute to pull the new image.
- Path filtering avoids redeploying the runtime for documentation-only changes.

Production improvement: GitHub OIDC + immutable image tags / digests.

9. Cost decisions and production trade-offs

The dev environment demonstrates patterns without pretending to be production-sized

Area	Capstone / dev choice	Production direction
Application capacity	1 desired EC2	2+ instances across AZs
NAT	Single NAT Gateway	Redundant NAT or workload-specific egress design
Ingress	HTTP ALB	HTTPS + ACM + custom domain
CI identity	Access keys in GitHub Secrets	GitHub OIDC federation
Image rollout	latest + instance refresh	Immutable versioned tags/digests + rollback

Primary idle-cost controls: small capacity + Terraform destroy when the demo environment is not required.

10. Demo and key takeaways

The application is small; the engineering lifecycle around it is the project

1. App + /health

2. TMDB live call

3. DynamoDB state

4. GitHub Actions

CloudMovie demonstrates:

- Infrastructure as Code
- Private networking and IAM-based security
- Managed persistence and secrets
- Self-healing and observability
- Automated application delivery

Engineering lesson

Use the minimum architecture that clearly solves the problem - and be explicit about every trade-off.

Questions